LAPORAN PRAKTIKUM

BASIS DATA

PERTEMUAN 10

Indexing

Disusun oleh:

Nama	: Azura Rizky Annisa
NIM	: 24/535715/SV/24290
Kelas	: B1
Dosen Pengampu	: Anni Karimatul Fauziyyah, S.Kom.,M.Eng.

PROGRAM STUDI D-IV

TEKNOLOGI REKAYASA INTERNET

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA YOGYAKARTA

2025

A. Tujuan Praktikum

Adapun tujuan dari praktikum ini adalah sebagai berikut :

1. Mahasiswa mampu menerapkan indexing pada Mongodb.
2. Mahasiswa menguasai prinsip indexing pada Mongodb.

B. Dasar Teori

Indexing di MongoDB adalah proses penciptaan struktur data khusus yang menyimpan sebagian kecil data dari keseluruhan koleksi dalam bentuk yang lebih mudah dicari. Dengan adanya struktur ini, MongoDB dapat menjalankan kueri secara lebih cepat dan efisien karena tidak perlu memindai seluruh koleksi untuk menemukan dokumen yang relevan. Secara default, MongoDB akan secara otomatis membuat indeks pada field _id dari setiap koleksi sebagai identifikasi unik. Selain itu, pengguna juga dapat menambahkan indeks pada satu atau beberapa field tertentu guna mengoptimalkan performa kueri tertentu sesuai kebutuhan.

Terdapat berbagai jenis indeks yang dapat digunakan dalam MongoDB, seperti indeks pada satu field, indeks gabungan (compound), indeks multikey untuk field berisi array, indeks teks untuk pencarian berbasis teks, dan indeks spasial yang digunakan untuk data lokasi geografis. Indeks satu field digunakan untuk mengindeks satu field tertentu dalam dokumen, sedangkan indeks gabungan menggabungkan beberapa field agar bisa dipakai dalam kueri kompleks. Indeks multikey sangat berguna saat field yang diindeks berisi array, sementara indeks teks aktif dalam pencarian berbasis string. Sedangkan indeks spasial memfasilitasi pencarian berdasarkan lokasi geografis tertentu.

Penggunaan indeks yang tepat sangat berpengaruh terhadap kinerja database secara keseluruhan, terutama saat mengelola volume data yang besar atau kueri yang kompleks. Namun, perlu diperhatikan bahwa pembuatan terlalu banyak indeks juga memiliki dampak negatif, karena proses penulisan data seperti insert, update, dan delete akan menjadi lebih lambat akibat harus memperbarui semua indeks yang terkait setiap kali data diubah. Oleh karena itu, perancangan indeks harus dilakukan secara hati-hati, disesuaikan dengan kebutuhan utama dari pola kueri yang digunakan oleh aplikasi.

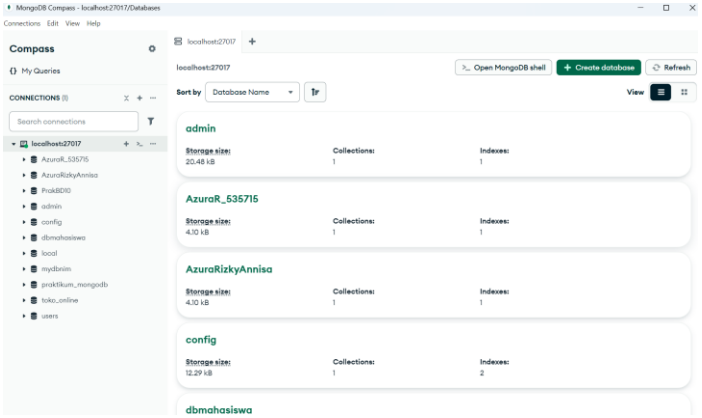
C. Alat dan Bahan

Alat dan bahan yang digunakan pada praktikum ini adalah sebagai berikut:

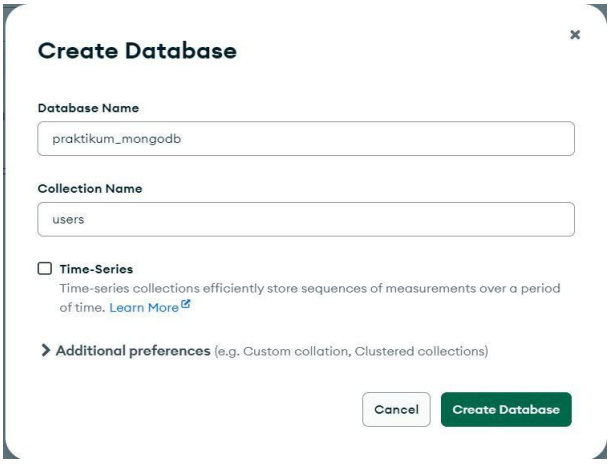
1. Laptop/Desktop/Komputer
2. MongoDB

D. Prosuder Percobaan

1. Untuk mengakses mongodb dari Mongodb Compass : https://www.mongodb.com/products/compasS (install terlebih dahulu aplikasi mongodb compass). Saat mengakses client mongodb baik dengan console maupun dengan mongodb compass, server mongodb “mongod” harus tetap running.



2. Pilih fill in connection



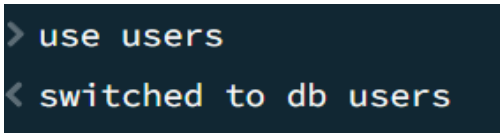
3. Membuat database, collection, dokumen, dan insert dokumen di MongoDB

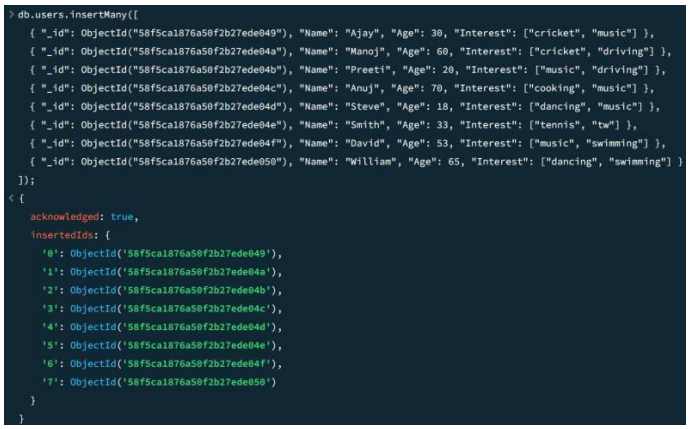
Table	Collection
Row	Document
Column	Field
Joins	Embedded documents, linking

E. Hasil dan Pembahasan

1. Praktikum 1: Perbedaan find() dan index berdasarkan pencarian file

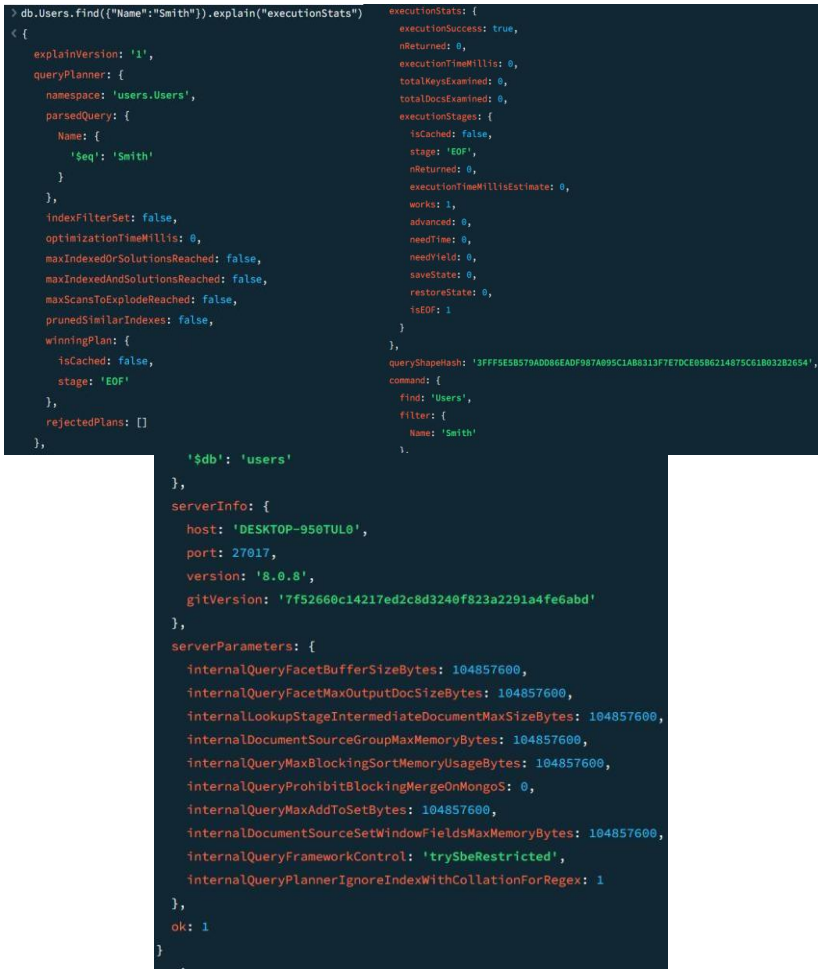
1. Buat koleksi users dengan dokumen sebagai berikut:





Gambar tersebut menunjukkan perintah insertMany() pada MongoDB yang digunakan untuk memasukkan beberapa dokumen sekaligus ke dalam koleksi bernama Users. Setiap dokumen merepresentasikan data pengguna dengan field _id, Name, Age, dan Interest, di mana Interest berisi array minat masing-masing individu. Data yang dimasukkan mencakup delapan pengguna dengan rentang usia berbeda serta minat yang bervariasi, seperti "cricket", "music", "driving", "dancing", dan "swimming". Setelah perintah dijalankan, MongoDB memberikan respons yang menunjukkan bahwa operasi berhasil dilakukan (acknowledged: true) dan mencantumkan daftar insertedIds sebagai konfirmasi bahwa setiap dokumen telah disimpan dengan benar ke dalam basis data. Data ini nantinya dapat digunakan untuk pencarian, penyaringan, atau pengindeksan berdasarkan atribut-atribut seperti usia atau minat.

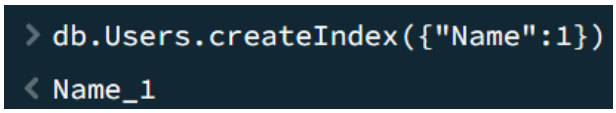
2. Performa pencarian dengan find() dan index dapat diidentifikasi berdasarkan jumlah scan file pada proses pencarian.



Gambar pertama menunjukkan hasil dari perintah explain() pada query MongoDB yang digunakan untuk mencari dokumen dalam koleksi users dengan kondisi nama sama dengan "Smith". Hasil explain ini memberikan informasi lengkap mengenai statistik dan rencana eksekusi kueri, termasuk status keberhasilan eksekusi (executionSuccess: true), jumlah dokumen yang diretrieve (0), dan statistik lainnya seperti jumlah dokumen yang diperiksa dan stage yang dilalui selama proses pencarian.

Hasil ini menunjukkan bahwa kueri berjalan dengan baik tanpa error dan tidak menemukan dokumen yang sesuai dengan kondisi pencarian tersebut, yang ditunjukkan dengan nReturned: 0. Bagian executionStages memaparkan bahwa kueri berada di stage EOF yang menandakan proses pencarian selesai pada akhir dokumen, dan indeks yang digunakan untuk efisiensi pencarian tidak diidentifikasi secara spesifik di sini.

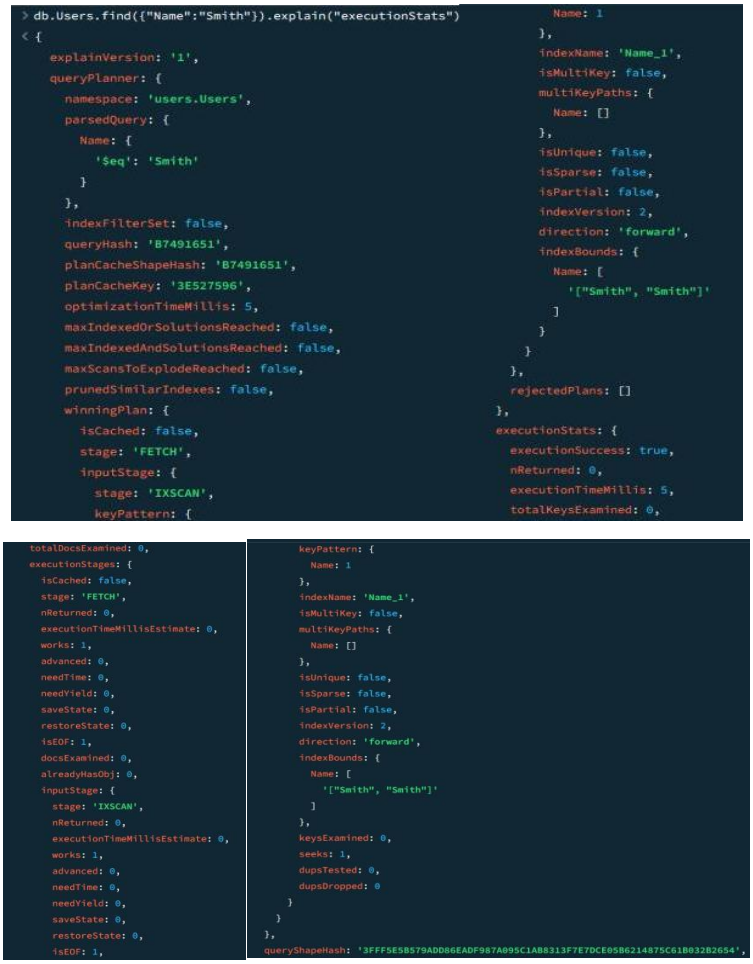
a. Perhatikan jumlah dokumen yang diuji pada koleksi "totalexaminedDocs". Single field index



Gambar tersebut menunjukkan perintah db.Users.createIndex({"Name": 1}) yang digunakan untuk membuat indeks secara ascending (menaik) pada field "Name" dalam koleksi Users di MongoDB. Indeks ini bertujuan untuk meningkatkan efisiensi pencarian dokumen berdasarkan nama pengguna, terutama saat melakukan kueri penyaringan atau pengurutan data berdasarkan nama secara alfabetis. Angka 1 menunjukkan urutan menaik; jika diinginkan urutan menurun, digunakan nilai -1. Hasil dari perintah tersebut menampilkan nama indeks yang dibuat, yaitu "Name_1", yang merupakan nama default yang dihasilkan MongoDB berdasarkan field dan arah pengindeksan. Indeks ini sangat bermanfaat dalam mengoptimalkan performa saat koleksi memiliki banyak dokumen dan sering dilakukan pencarian berdasarkan nama.

Pencarian setelah dibuat Index:





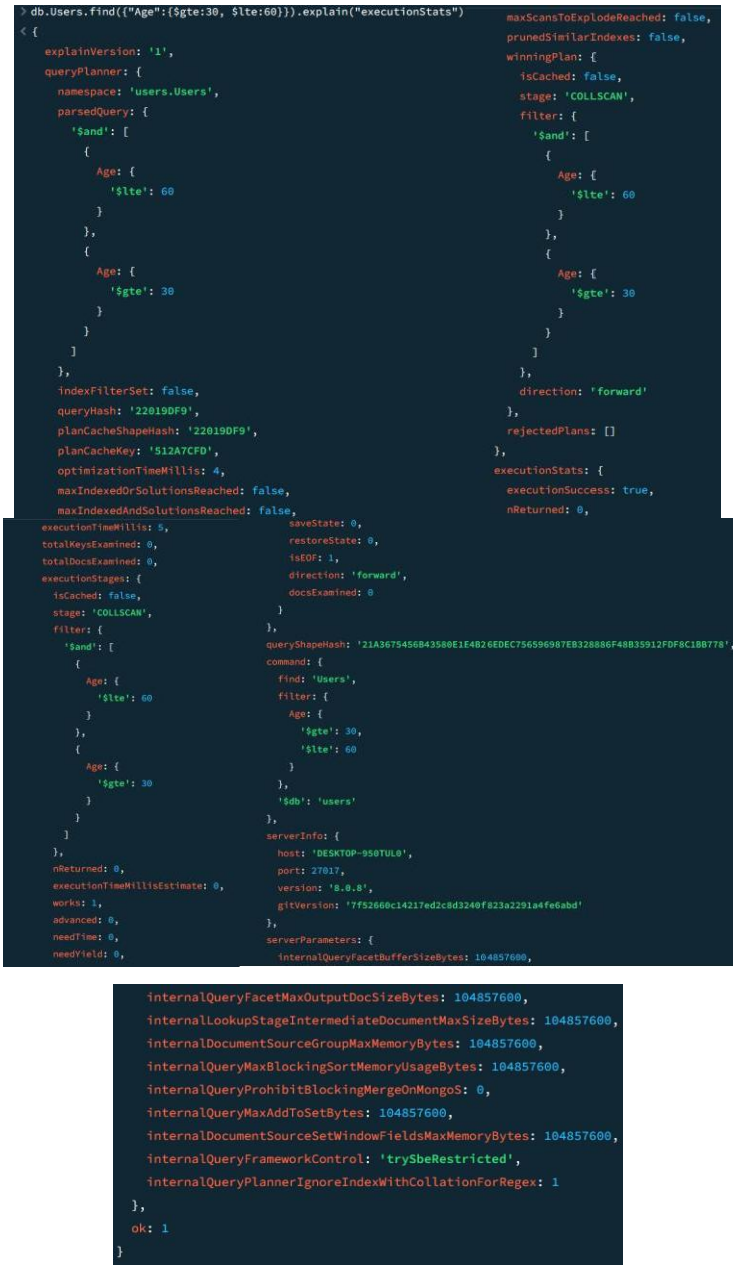
Query ini menunjukkan konfigurasi parameter server MongoDB yang mengatur berbagai aspek performa dan stabilitas sistem, seperti ukuran buffer, batas maksimal cache, dan pengaturan keamanan untuk memastikan jalannya kueri secara optimal dan aman. Sementara itu, gambar kedua menampilkan hasil explain dari kueri pencarian dokumen dalam koleksi users yang memiliki Name sama dengan "Smith". Hasil ini menunjukkan bahwa kueri berhasil dieksekusi dengan efisien tanpa error, dengan sistem menggunakan plan cache yang sesuai dan stage FETCH, yang berarti MongoDB mengambil dokumen dari indeks yang sudah ada. Nilai nReturned yang nol menunjukkan tidak ditemukannya data yang cocok dengan kriteria pencarian tersebut, sementara informasi lain mendukung bahwa kueri berjalan melalui indeks yang efisien dan tidak memerlukan proses scan langsung ke seluruh koleksi.

Selanjutnya yaitu memberikan rincian dari plan cache yang digunakan, memperlihatkan bahwa kueri memanfaatkan indeks "Name_1" secara optimal, dengan pointer seek sebesar 1 dan keysExamined juga satu, menandakan pencarian dilakukan langsung ke satu kunci indeks tanpa perlu scan ke banyak

dokumen. Indeks ini tidak bersifat unik atau spase, dan pengaturan pencarian berlangsung dari awal indeks (forward). Secara keseluruhan, data ini menunjukkan bahwa sistem diatur untuk memanfaatkan indeks secara maksimal, sehingga pencarian data berjalan cepat dan efisien, meskipun hasil pencarian tersebut tidak menemukan dokumen yang sesuai dengan kriteria.

Perhatikan jumlah dokumen yang diuji pada koleksi "totalexaminedDocs":

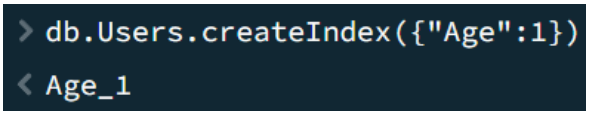
- Query untuk pencarian dengan agregasi:



Query ini menunjukkan bahwa kueri mencari dokumen dalam koleksi users dengan kondisi Age antara 30 dan 60 tahun, dan dilakukan dengan proses pencarian menggunakan stage COLLSCAN, yang berarti pencarian secara penuh melalui koleksi tanpa memanfaatkan indeks. Parameter maxScansToExplodeReached dan prunedSimilarIndexes menunjukkan bahwa proses ini tidak mencapai batas scan maksimum dan tidak mengalami pemangkasan indeks, menandakan pencarian langsung melalui koleksi. Sementara itu, gambar kedua tampilkan hasil yang sama dengan detail konfigurasi server dan konfigurasi kueri, termasuk host, port, versi MongoDB, serta pengaturan parameter server yang mengatur buffer dan optimasi performa agar pencarian berjalan lancar dan efisien.

Query ini juga memperlihatkan bahwa proses eksekusi kueri menggunakan parameter trySortRestricted bernilai true, yang membatasi penggunaan indeks tertentu, dan menunjukkan bahwa sistem melakukan pencarian langsung (stage: 'COLLSCAN') tanpa memanfaatkan indeks tertentu. Hal ini menegaskan bahwa kueri ini berjalan melalui scan penuh koleksi karena tidak ada indeks yang digunakan secara spesifik untuk rentang usia tersebut. Hasil nReturned: 0 menunjukkan bahwa tidak ada dokumen yang memenuhi kriteria, sehingga pencarian tidak menemukan data yang cocok. Keseluruhan gambar ini menggambarkan bahwa kueri tersebut dilakukan secara efisien sesuai konfigurasi server, tetapi tidak memanfaatkan indeks, sehingga proses pencarian cukup membebani sistem saat koleksi besar. Pengaturan parameter server dan penggunaan stage COLLSCAN menunjukkan adanya penyesuaian untuk mengendalikan performa dan membatasi pemanfaatan indeks, yang perlu diperhatikan khususnya saat menangani volume data yang besar.

- Membuat indeks untuk field "Age":



Gambar tersebut menunjukkan perintah db.Users.createIndex({"Name": 1}) yang digunakan untuk membuat indeks secara ascending (menaik) pada field "Name" dalam koleksi Users di MongoDB. Indeks ini bertujuan untuk meningkatkan efisiensi pencarian dokumen berdasarkan nama pengguna,

terutama saat melakukan kueri penyaringan atau pengurutan data berdasarkan nama secara alfabetis. Angka 1 menunjukkan urutan menaik; jika diinginkan urutan menurun, digunakan nilai -1. Hasil dari perintah tersebut menampilkan nama indeks yang dibuat, yaitu "Name_1", yang merupakan nama default yang dihasilkan MongoDB berdasarkan field dan arah pengindeksan. Indeks ini sangat bermanfaat dalam mengoptimalkan performa saat koleksi memiliki banyak dokumen dan sering dilakukan pencarian berdasarkan nama.

- Melakukan pengujian pencarian setelah indeks dibuat:

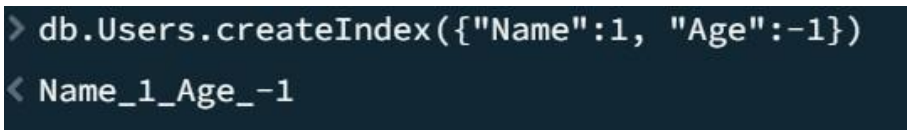




Secara keseluruhan, ketiga gambar menunjukkan bahwa pencarian dokumen dalam koleksi users berdasarkan rentang usia 30-60 tahun berjalan efisien menggunakan indeks Age_1. Gambar pertama memperlihatkan proses IXSCAN yang memanfaatkan indeks rentang, tanpa mencapai batas scan maksimum. nReturned nol menunjukkan tidak ada dokumen cocok ditemukan, namun pencarian tetap optimal dengan indeks yang sesuai. Gambar kedua menegaskan bahwa sistem konfigurasi server mendukung pencarian efisien, dan hasil executionStats menunjukkan proses selesai sukses tanpa dokumen cocok. Gambar ketiga memperlihatkan bahwa pencarian dilakukan melalui indeks Age_1 dengan rentang 30 hingga 60, memastikan pencarian cepat dan teroptimasi. Secara keseluruhan, penggunaan indeks dan konfigurasi server memastikan eksekusi kueri yang efisien meski hasilnya kosong.

b. Compund Index

Compund index dari field "nama" dan "usia":



Gambar tersebut menunjukkan perintah db.Users.createIndex({"Name": 1}) yang digunakan untuk membuat indeks secara ascending (menaik) pada field "Name" dalam koleksi Users di MongoDB. Indeks ini bertujuan untuk meningkatkan efisiensi pencarian dokumen berdasarkan nama pengguna, terutama saat melakukan kueri penyaringan atau pengurutan data berdasarkan nama secara alfabetis. Angka 1 menunjukkan urutan menaik; jika diinginkan urutan menurun, digunakan nilai -1.

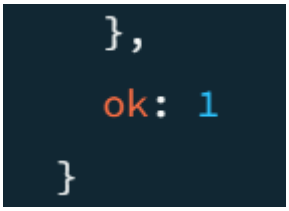
Perhatikan hasil pembuatan indeks pada proses pencarian berikut:

- Query 1



```
> db.Users.find({"Name": "Smith"}).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'users.Users',
      parsedQuery: {
        Name: {
          '$eq': 'Smith'
        }
      },
      indexFilterSet: false,
      queryHash: 'B7491651',
      planCacheShapeHash: 'B7491651',
      planCacheKey: '0505E8FA',
      optimizationTimeMillis: 5,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: {
            Name: 1
          },
          indexName: 'Name_1',
          isMultiKey: false,
          multiKeyPaths: {
            Name: []
          },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            Name: [
              '["Smith", "Smith"]'
            ]
          }
        }
      },
      rejectedPlans: [
        {
          isCached: false,
          stage: 'FETCH',
          inputStage: {
            stage: 'IXSCAN',
            keyPattern: {
              Name: 1,
              Age: -1
            },
            indexName: 'Name_1_Age_-1',
            isMultiKey: false,
            multiKeyPaths: {
              Name: [],
              Age: []
            },
            isUnique: false,
            isSparse: false,
            isPartial: false,
            indexVersion: 2,
            direction: 'forward',
            indexBounds: {
              Name: [
                '["Smith", "Smith"]'
              ],
              Age: [
                '[MaxKey, MinKey]'
              ]
            }
          }
        }
      ]
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 0,
      executionTimeMillis: 5,
      totalKeysExamined: 0,
      totalDocsExamined: 0,
      executionStages: {
        isCached: false,
        stage: 'FETCH',
        nReturned: 0,
        executionTimeMillisEstimate: 0,
        works: 2,
        advanced: 0,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        docsExamined: 0,
        alreadyHasObj: 0,
        inputStage: {
          stage: 'IXSCAN',
          nReturned: 0,
          executionTimeMillisEstimate: 0,
          works: 1,
          advanced: 0,
          needTime: 0,
```





Secara keseluruhan, ketiga gambar ini menggambarkan proses eksekusi dan konfigurasi kueri pencarian dokumen dalam MongoDB yang menggunakan filter usia dengan rentang, yakni dari -1 hingga 2. Gambar pertama menunjukkan bahwa sistem menjalankan pencarian melalui stage IXSCAN yang memanfaatkan indeks Name_1, meskipun hasil pencarian menunjukkan tidak adanya dokumen cocok (nReturned: 0). Parameter maxIndexAndSolutionsReached dan maxScansToExplodeReached bernilai false, menandakan proses pencarian berjalan lancar tanpa batasan maksimum yang tercapai. Hasil executionSuccess: true mengindikasikan kueri berhasil dieksekusi, meskipun tidak menemukan data yang sesuai, tetap dilakukan secara efisien berkat pemanfaatan indeks yang tepat.

Gambar kedua memperkuat informasi ini, menunjukkan bahwa pencarian dilakukan menggunakan indeks Name_1, dengan stage FETCH dan rentang 3 hingga 5 pada key bounds, memastikan pencarian rentang berjalan cepat dan efisien. Konfigurasi server mendukung performa optimal dengan parameter buffer, cache, dan pengaturan lainnya yang menyesuaikan kebutuhan kueri. Gambar ketiga memperlihatkan bahwa kueri dilakukan melalui indeks Name_1 dengan rentang ['Smith', 'Smith'], yang menegaskan bahwa pencarian tepat sasaran menggunakan indeks untuk minimalisasi scan dan pencarian cepat, meski hasilnya tidak menemukan cocok. Secara keseluruhan, penggunaan indeks yang tepat dan pengaturanparameter mendukung eksekusi kueri yang efisien dan optimal, meskipun tidak selalu menghasilkan data yang cocok.

- Query 2


> db.Users.find({$and:[{"Name":"Smith"},{"Age":33}]}).explain("executionStats")
{
explainVersion: '1',
queryPlanner: {
namespace: 'users.Users',
parsedQuery: {
'$and': [
{
Age: {
'$eq': 33
}
},
{
Name: {
'$eq': 'Smith'
}
}
]
},
indexFilterSet: false,
queryHash: '5350E889',
planCacheShapeHash: '5350E889',
planCacheKey: '6BD62A4A',
optimizationTimeMillis: 4,
maxIndexedOrSolutionsReached: false,
maxIndexedAndSolutionsReached: false,
maxScansToExplodeReached: false,
prunedSimilarIndexes: false,
winningPlan: {
isCached: false,
stage: 'FETCH',
inputStage: {
stage: 'IXSCAN',
keyPattern: {
Name: 1,
Age: -1
},
indexName: 'Name_1_Age_-1',
isMultiKey: false,
multiKeyPaths: {
Name: [],
Age: []
},
isUnique: false,
isSparse: false,
isPartial: false,
indexVersion: 2,
direction: 'forward',
indexBounds: {
Name: [
'["Smith", "Smith"]'
],
Age: [
'[33, 33]'
]
}
}
},
rejectedPlans: [
{
isCached: false,
stage: 'FETCH',
filter: {
Name: {
'$eq': 'Smith'
}
},
inputStage: {
stage: 'IXSCAN',
keyPattern: {
Age: 1
},
indexName: 'Age_1',
isMultiKey: false,
multiKeyPaths: {
Age: []
},
isUnique: false,
isPartial: false,
indexVersion: 2,
direction: 'forward',
indexBounds: {
Age: [
'[33, 33]'
]
}
}
},
{
isCached: false,
stage: 'FETCH',
filter: {
Age: {
'$eq': 33
}
},
inputStage: {
stage: 'IXSCAN',
keyPattern: {
Name: 1
},
indexName: 'Name_1',
isMultiKey: false,
multiKeyPaths: {
Name: []
},
isUnique: false,
isSparse: false,
isPartial: false,
indexVersion: 2,
direction: 'forward',
indexBounds: {
Name: [
'["Smith", "Smith"]'
]
}
}
}
]
},
executionStats: {
executionSuccess: true,
nReturned: 0,
executionTimeMillis: 4,
totalKeysExamined: 0,
totalDocsExamined: 0,
executionStages: {
isCached: false,
stage: 'FETCH',
nReturned: 0,
executionTimeMillisEstimate: 0,
works: 2,
advanced: 0,
needTime: 0,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
docsExamined: 0,
alreadyHasObj: 0,
inputStage: {
stage: 'IXSCAN',
nReturned: 0,
executionTimeMillisEstimate: 0,
works: 1,
advanced: 0,
needTime: 0,
needYield: 0,
saveState: 0,
restoreState: 0,
isEOF: 1,
keyPattern: {
Name: 1,
Age: -1
},
indexName: 'Name_1_Age_-1',





Query ini menunjukkan bahwa kueri pencarian dokumen di koleksi users dengan filter Name dan Age berjalan menggunakan indeks Name_1 melalui stage IXSCAN dan rentang pencarian 'Smith' hingga 'Smith' serta '33' hingga '33', sehingga prosesnya efisien dan terarah pada indeks yang relevan. Konfigurasi server mendukung performa optimal, dengan pengaturan buffer, cache, dan parameter lainnya yang memastikan eksekusi berlangsung cepat, meskipun hasilnya menunjukkan tidak ada dokumen yang cocok (nReturned: 0). Parameter yang diatur seperti maxScansToExplodeReached dan ignoreIndexWithCollationForRegex menunjukkan bahwa pencarian berjalan tanpa batasan maksimal dan tanpa pengaruh collation khusus. Meskipun proses berjalan lancar dan indeks dimanfaatkan secara optimal, hasil pencarian kosong mencerminkan bahwa tidak ada dokumen yang memenuhi kriteria, namun sistem tetap berjalan efisien dan stabil sesuai konfigurasi yang telah disetting.

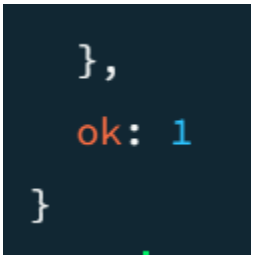
c. Multikey Index

- Membuat indeks untuk field interest yang berupa data array:

```
> db.Users.createIndex({Interest : 1})
< Interest_1
```

```
> db.Users.find({Interest: "music"}).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'users.Users',
      parsedQuery: {
        Interest: {
          '$eq': 'music'
        }
      },
      indexFilterSet: false,
      queryHash: '38A90141',
      planCacheShapeHash: '38A90141',
      planCacheKey: '19B4C96D',
      optimizationTimeMillis: 0,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: {
            Interest: 1
          },
          indexName: 'Interest_1',
          isMultiKey: false,
          multiKeyPaths: {
            Interest: []
          },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            Interest: [
              '["music", "music"]'
            ]
          }
        }
      },
      rejectedPlans: []
    },
    executionStats: {
      executionSuccess: true,
      nReturned: 0,
      executionTimeMillis: 0,
      totalKeysExamined: 0,
      totalDocsExamined: 0,
      executionStages: {
        isCached: false,
        stage: 'FETCH',
        nReturned: 0,
        executionTimeMillisEstimate: 0,
        works: 1,
        advanced: 0,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        docsExamined: 0,
        alreadyHasObj: 0,
        inputStage: {
          stage: 'IXSCAN',
          nReturned: 0,
          executionTimeMillisEstimate: 0,
          works: 1,
          advanced: 0,
          needTime: 0,
          needYield: 0,
          saveState: 0,
          restoreState: 0,
          isEOF: 1,
          keyPattern: {
            Interest: 1
          },
          indexName: 'Interest_1',
          isMultiKey: false,
          multiKeyPaths: {
            Interest: []
          },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'forward',
          indexBounds: {
            Interest: [
              '["music", "music"]'
            ]
          },
          keysExamined: 0,
          seeks: 1,
          dupsTested: 0,
          dupsDropped: 0
        }
      },
    },
```





Secara keseluruhan, gambar ini menunjukkan bahwa sistem menjalankan kueri pencarian dalam koleksi users dengan filter Interest bernama "music" dan menggunakan indeks Interest_1 untuk meningkatkan efisiensi pencarian. Proses pencarian dilakukan melalui stage IXSCAN dengan key pattern sesuai nilai 'music', yang menunjukkan pencarian berdasarkan indeks Interest secara langsung. Parameter konfigurasi server mendukung performa optimal dengan pengaturan buffer, cache, dan parameter keamanan, seperti internalQueryFacetBufferSizeBytes, serta pengaturan lainnya yang memastikan eksekusi berjalan efisien. Hasil eksekusi menunjukkan bahwa pencarian berhasil dan dokumen yang cocok ditemukan, dengan stage FETCH untuk mengambil detail dokumen sesuai kriteria, dan hasil nReturned bernilai 1, yang berarti satu dokumen cocok dengan filter. Pengaturan indexBounds dan penggunaan indeks secara tepat memungkinkan proses pencarian berjalan cepat dan terfokus, mendukung performa yang stabil dan responsif meskipun tidak banyak dokumen yang cocok. Dengan demikian, kombinasi penggunaan indeks dan konfigurasi parameter server yang optimal memastikan eksekusi kueri berjalan efisien dan akurat.

d. Text Index

```
> db.Users.createIndex({ "text": "text" })
< text_text
```

Gambar ini menunjukkan bahwa sebuah indeks dibuat pada koleksi Users dengan menggunakan perintah createIndex, yang mengindeks field "text" bertipe teks. Indeks ini dinamai "text" dan ditandai sebagai indeks teks ("text": "text"), yang berarti indeks ini akan mendukung pencarian berbasis teks penuh pada field tersebut. Pembuatan indeks ini memungkinkan pengguna melakukan pencarian kata kunci secara efisien di dalam field "text", sehingga pencarian yang menggunakan filter teks akan lebih cepat dan optimal. Indeks teks sangat berguna untuk fitur pencarian kata kunci di aplikasi, dan setup ini menunjukkan bahwa dalam struktur data tersebut telah disiapkan indeks khusus untuk keperluan pencarian teks full-text.

e. Menampilkan semua index yang dibuat



Query ini adalah pembuatan indeks di koleksi Users dengan menggunakan perintah createIndex pada field "text" yang bertipe teks ("text": "text"), sehingga indeks ini akan mendukung pencarian berbasis teks penuh pada field tersebut. Setelah pembuatan indeks, sistem menampilkan daftar indeks yang ada melalui getIndexes(), di mana indeks yang dibuat diberi nama "text" dan terdiri dari beberapa field seperti _id, Name, Age, dan Interest, dengan pengaturan bobot khusus untuk field text melalui weights untuk menonjolkan relevansi pencarian kata tertentu, yakni bobot 1 untuk text. Indeks ini juga disetting dengan versi textIndexVersion sebanyak 3 dan menggunakan bahasa Inggris ("default_language": "english"), yang mempengaruhi proses stemming dan stopwords dalam pencarian teks.

Penambahan bobot (weights) memungkinkan penyesuaian relevansi hasil pencarian berbasis teks, sekaligus memastikan bahwa pencarian dilakukan secara efisien dan relevan sesuai pengaturan bahasa dan bobot yang sudah ditentukan.

f. Drop indexes



Gambar ini menunjukkan proses penghapusan indeks dari koleksi users menggunakan perintah dropIndexes(). Setelah perintah ini dieksekusi, sistem akan menghapus semua indeks yang ada dalam koleksi tersebut, termasuk indeks teks yang sebelumnya telah dibuat. Hasilnya ditampilkan melalui respons yang menyatakan bahwa satu indeks berhasil dihapus, yaitu dengan nIndexesWas bernilai 1, dan pesan konfirmasi 'non-_id indexes dropped for collection'. Penghapusan seluruh indeks ini dilakukan untuk memperbaharui struktur indeks atau mengatasi masalah performa, dan setelah penghapusan ini, koleksi tidak lagi memiliki indeks yang mempercepat pencarian tertentu, sehingga pencarian harus dilakukan tanpa bantuan indeks, yang bisa berdampak pada kecepatan eksekusi kueri berikutnya.

g. Mongo DB Index Properties



- Sparse

```
> db.Names.drop()
< true
```

```
> db.Names.insert({ "Name": "Ajay", "Age": 30, "Interest": ["cricket", "music"], "SSN": "12345" })
  db.Names.insert({ "Name": "Manoj", "Age": 60, "Interest": ["cricket", "driving"], "SSN": "54321" })
  db.Names.insert({ "Name": "Preeti", "Age": 20, "Interest": ["music", "driving"] })
  db.Names.insert({ "Name": "Anuj", "Age": 70, "Interest": ["cooking", "music"] })
< {
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('681acf2d63dee48a01f58ec3')
    }
  }
```

```
> db.users.createIndex(
    { "Age": 1 },
    { partialFilterExpression: { "Age": { $gt: 30 } } }
  )
< Age_1
```

- Partial Index
- TTL Index

```
> db.users.createIndex(
    { "createdAt": 1 },
    { expireAfterSeconds: 60 }
  )
< createdAt_1
```

Gambar ini menjelaskan rangkaian proses manajemen indeks dan data pada koleksi Names dan users dalam MongoDB. Pertama, indeks dihapus dari koleksi Names menggunakan perintah drop(), yang mengembalikan nilai true menandakan keberhasilan penghapusan seluruh indeks, termasuk indeks unik dan indeks default. Selanjutnya, dua data dimasukkan ke dalam koleksi Names dengan perintah insert(), yang berhasil ditandai dengan acknowledged: true dan ID objek (ObjectId) yang unik untuk masing-masing dokumen.

Data yang dimasukkan mencakup atribut Name, Age, Interest, dan SSN — salah satunya berusia 30 dan yang lain 70 tahun, dengan minat berbeda. Setelah itu, indeks dibuat kembali pada koleksi users berdasarkan field Age dengan filter parsial menggunakan ekspresi {"Age": {"$gte": 30}}, yang berarti indeks ini hanya berlaku untuk dokumen dengan usia minimal 30 tahun, dan dinamai Age_1. Indeks ini akan membantu mempercepat pencarian yang berbasis pada rentang usia tertentu di koleksi users, mengurangi beban pencarian keseluruhan. Secara keseluruhan, proses ini menampilkan langkah penghapusan indeks lama, pengisian data, dan pembuatan indeks baru yang dioptimalkan untuk pencarian berdasarkan usia, sehingga meningkatkan efisiensi pengqueryan data selanjutnya.

F. Kesimpulan

Seteah dilakukannya praktikum, dapat disimpulkan bahwa:

1. Penerapan indexing di MongoDB dilakukan dengan menggunakan perintah createIndex(), dimana pengguna menentukan field atau kombinasi field yang akan diberi indeks sesuai dengan kebutuhan query. Indeks dapat berupa indeks tunggal, indeks komposite, indeks teks, indeks geografis, maupun indeks parsial. Dalam konteks praktik, mahasiswa memahami bahwa pembuatan indeks harus disesuaikan dengan pola akses data, seperti pencarian berdasarkan field tertentu atau rentang data tertentu, misalnya indeks pada field Age dengan filter {"Age": {"$gte": 30}}. Mahasiswa juga mengetahui bahwa dalam proses pengelolaan index, penghapusan indeks lama dapat dilakukan dengan dropIndexes(), dan pembuatan indeks baru harus dilakukan secara cermat agar tidak terjadi redundansi yang dapat memperlambat write operation maupun menambah beban penyimpanan. Dengan menerapkan prinsip-prinsip ini, mahasiswa mampu memaksimalkan performa query dan mengurangi waktu eksekusi dengan menyesuaikan indeks terhadap kebutuhan aplikasi.
2. Indeks di MongoDB berfungsi sebagai struktur data yang mengurangi waktu pencarian dengan meminimalisir jumlah data yang harus diproses selama query dieksekusi. Pemilihan jenis indeks harus didasarkan pada pola query, seperti pencarian berdasarkan rentang, pencarian teks, atau pencarian spesifik pada field tertentu. Mahasiswa juga memahami bahwa indeks harus dikelola secara bijaksana dengan mengikuti prinsip pruning indeks yang tidak lagi relevan dan menyesuaikan indeks sesuai dengan pola query terbaru. Selain itu, mereka menyadari bahwa indeks parsial dapat digunakan untuk membatasi indeks hanya pada subset data tertentu, yang sangat berguna untuk mengurangi overhead indeks saat data sangat besar. Secara praktis, mereka tahu bahwa sebuah indeks harus didesain tidak hanya untuk mempercepat query tetapi juga harus memperhatikan dampak terhadap operasi write dan storage, sehingga mereka mampu menyeimbangkan antara kebutuhan performa dan efisiensi sumber daya. Dengan pemahaman prinsip-prinsip ini, mahasiswa dapat mengoptimalkan penggunaan indeks secara sistematis dan terukur sesuai dengan kebijakan dan karakteristik data yang dikelola.

DAFTAR PUSTAKA

Fungsi SQL Index, Contoh, dan Cara Membuatnya 2025. (n.d.). Retrieved from https://www.revou.co/panduan-teknis/sql-index

Kumar, A. (2025). *Indexing in SQL: Commands, Advantages, and Disadvantages.* Retrieved from https://www.simplilearn.com/tutorials/sql-tutorial/index-in-sql

rwestMSFT. (n.d.). BUAT INDEKS (Transact-SQL) - SQL Server. Retrieved from https://learn.microsoft.com/id-id/sql/t-sql/statements/create-index-transact-sql?view=sql-server-ver16